



**KG COLLEGE OF ARTS AND SCIENCE**

Autonomous Institution | Affiliated to Bharathiar University

Accredited with A++ Grade by NAAC

ISO 9001:2015 Certified Institution

**KGiSL** Campus, Saravanampatti, Coimbatore - 641 035

## LAB MANUAL

**Agentic AI & Intelligent Systems Core Lab -V**

**V Semester**

**B.Sc. CS (AI & DS)**

**2024 BATCH**

**Course Code: 24BAI52P**

**Course Name Lab : Agentic AI & Intelligent Systems Core Lab -V**

**Category Hours /Week: 5**

**Credits: 3**

**Core Lab - V: Agentic AI & Intelligent Systems**

| <b>S.No.</b> | <b>List of Programs</b>                                                          | <b>Page No</b> |
|--------------|----------------------------------------------------------------------------------|----------------|
| 1            | Implement basic search strategies – 8-Puzzle, 8 - Queens problem, Cryptarithmic. | 2              |
| 2            | Implement A* and memory bounded A* algorithms                                    | 7              |
| 3            | Implement Mini-max algorithm for game playing (Alpha-Beta pruning)               | 15             |
| 4            | Solve constraint satisfaction problems                                           | 17             |
| 5            | Implement propositional model checking algorithms                                | 20             |
| 6            | Implement forward chaining, backward chaining, and resolution strategies         | 22             |
| 7            | Build naïve Bayes models                                                         | 27             |
| 8            | Implement Bayesian networks and perform inferences                               | 29             |
| 9            | Design and Implementation of a Goal-Based Autonomous Agent                       | 31             |
| 10           | Implementation of Reflex Agent and Model-Based Agent                             | 34             |
| 11           | Simulation of Environment-Aware Intelligent Agent                                | 38             |
| 12           | Implementation of a Learning Agent with Memory Update                            | 41             |
| 13           | Simulation of Cooperative Multi-Agent Communication System                       | 45             |
| 14           | Trust & Reputation System - Skill: Evaluation, Connectors - Online Rating System | 48             |
| 15           | Design of Distributed Problem-Solving Agents                                     | 51             |

**Ex:No:1.****Basic Search Strategies****1a. 8-Puzzle Problem (Breadth-First Search)****Aim**

To find the shortest sequence of moves that transforms a given initial 3 times, 3 grid configuration of 8 numbered tiles into a specific sequential goal state, utilizing the A\* Search algorithm with the Manhattan Distance heuristic.

**Algorithm**

1. Define the Goal State and a method to calculate the Manhattan Distance (h) for any given state (the sum of absolute horizontal and vertical distances of all tiles from their correct goal positions).
2. Create an Open Set (a priority queue) to store states to be evaluated, and a Closed Set (a set) to track already visited states.
3. Initialize the Open Set with the starting configuration, setting its path cost ( $g = 0$ ) and total estimated cost ( $f = g + h$ ).
4. While the Open Set is not empty:
  - Pop the state with the lowest f-score from the queue. This is the `current_state`.
  - If `current_state` matches the Goal State, return the accumulated path sequence as the solution.
  - Add `current_state` to the Closed Set.
5. For each valid neighbor generated by moving the blank space (0) Up, Down, Left, or Right:
  - If the neighbor is already in the Closed Set, ignore it.
  - Else, calculate its new path cost ( $g_{\text{new}} = g_{\text{current}} + 1$ ) and total cost ( $f_{\text{new}} = g_{\text{new}} + h_{\text{neighbor}}$ ).
  - Push the neighbor alongside its calculated costs and updated history path into the Open Set priority queue.
6. If the Open Set becomes empty without hitting the Goal State, return that no solution exists.

**Python Program****1(A). 8-Puzzle (Uninformed Search – BFS)**

```
from collections import deque
```

```
goal = [1,2,3,4,5,6,7,8,0]
```

```
def get_neighbors(state):
    neighbors = []
    i = state.index(0)
    moves = [(-1,0),(1,0),(0,-1),(0,1)]
    x, y = divmod(i, 3)

    for dx, dy in moves:
        nx, ny = x+dx, y+dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new = state[:]
            ni = nx*3 + ny
            new[i], new[ni] = new[ni], new[i]
            neighbors.append(new)
    return neighbors

def bfs(start):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]

        visited.add(tuple(state))
        for n in get_neighbors(state):
            if tuple(n) not in visited:
                queue.append((n, path + [state]))

start = [1,2,3,4,0,5,6,7,8]
print(bfs(start))
```

### OUTPUT

```
[[1, 2, 3, 4, 0, 5, 6, 7, 8], [1, 2, 3, 4, 5, 0, 6, 7, 8], [1, 2, 3, 4, 5, 8, 6, 7, 0], [1, 2, 3, 4, 5, 8, 6, 0, 7],
[1, 2, 3, 4, 5, 8, 0, 6, 7], [1, 2, 3, 0, 5, 8, 4, 6, 7], [1, 2, 3, 5, 0, 8, 4, 6, 7], [1, 2, 3, 5, 6, 8, 4, 0, 7],
[1, 2, 3, 5, 6, 8, 4, 7, 0], [1, 2, 3, 5, 6, 0, 4, 7, 8], [1, 2, 3, 5, 0, 6, 4, 7, 8], [1, 2, 3, 0, 5, 6, 4, 7, 8],
[1, 2, 3, 4, 5, 6, 0, 7, 8], [1, 2, 3, 4, 5, 6, 7, 0, 8], [1, 2, 3, 4, 5, 6, 7, 8, 0]]
```

### Result

**The 8-Puzzle Problem was successfully solved using the Breadth-First Search (BFS) algorithm. The algorithm explored the puzzle level by level and found the shortest sequence**

of moves from the initial state to the goal state, demonstrating the completeness and optimality of BFS for unweighted search problems.

### 1b. 8-Queens Problem (Backtracking)

- **Aim:** Place eight non-attacking queens on an  $(8 \times 8)$  chessboard using a standard recursive depth-first backtracking strategy.
- **Algorithm:**
  1. Start in the leftmost column (index 0).
  2. If all columns contain a validated queen configuration, return true.
  3. For each row in the current column, verify row and diagonal safety constraints.
  4. If the position is safe, mark the cell with a queen (1).
  5. Recursively attempt to place queens in subsequent columns.
  6. If the downstream configuration fails, clear the cell (0) and backtrack.

#### Python Code

```
def is_safe(board, row, col):
    for i in range(row):
        if board[i] == col or abs(board[i] - col) == abs(i - row):
            return False
    return True

def solve_n_queens(n):
    board = [-1]*n

    def backtrack(row):
        if row == n:
            print(board)
            return
        for col in range(n):
            if is_safe(board, row, col):
                board[row] = col
                backtrack(row+1)

    backtrack(0)
```

solve\_n\_queens(8)

#### OUTPUT

```
[0, 4, 7, 5, 2, 6, 1, 3][0, 5, 7, 2, 6, 3, 1, 4][0, 6, 3, 5, 7, 1, 4, 2][0, 6, 4, 7, 1, 3, 5, 2][1, 3, 5, 7, 2, 0, 6, 4][1, 4, 6, 0, 2, 7, 5, 3][1, 4, 6, 3, 0, 7, 5, 2][1, 5, 0, 6, 3, 7, 2, 4][1, 5, 7, 2, 0, 3, 6, 4][1, 6, 2, 5, 7, 4, 0, 3][1, 6, 4, 7, 0, 3, 5, 2][1, 7, 5, 0, 2, 4, 6, 3][2, 0, 6, 4, 7, 1, 3, 5][2, 4, 1, 7, 0, 6, 3, 5][2, 4, 1, 7, 5, 3, 6, 0][2, 4, 6, 0, 3, 1, 7, 5][2, 4, 7, 3, 0, 6, 1, 5][2, 5, 1, 4, 7, 0, 6, 3][2, 5, 1, 6, 0, 3, 7, 4][2, 5, 1, 6, 4, 0, 7, 3][2, 5, 3, 0, 7, 4, 6, 1][2, 5, 3, 1, 7, 4, 6, 0][2, 5, 7, 0, 3, 6, 4, 1][2, 5, 7, 0, 4, 6, 1, 3][2, 5, 7, 1, 3, 0, 6, 4][2, 6, 1, 7, 4, 0, 3, 5][2, 6, 1, 7, 5, 3, 0, 4][2, 7, 3, 6, 0, 5, 1, 4][3, 0, 4, 7, 1, 6, 2, 5][3, 0, 4, 7, 5, 2, 6, 1][3, 1, 4, 7, 5, 0, 2, 6][3, 1, 6, 2, 5, 7, 0, 4][3, 1, 6, 2, 5, 7, 4, 0][3, 1, 6, 4, 0, 7, 5, 2][3, 1, 7, 4, 6, 0, 2, 5][3, 1, 7, 5, 0, 2, 4, 6][3, 5, 0, 4, 1, 7, 2, 6][3, 5, 7, 1, 6, 0, 2, 4][3, 5, 7, 2, 0, 6, 4, 1][3, 6, 0, 7, 4, 1, 5, 2][3, 6, 2, 7, 1, 4, 0, 5][3, 6, 4, 1, 5, 0, 2, 7][3, 6, 4, 2, 0, 5, 7, 1][3, 7, 0, 2, 5, 1, 6, 4][3, 7, 0, 4, 6, 1, 5, 2][3, 7, 4, 2, 0, 6, 1, 5][4, 0, 3, 5, 7, 1, 6, 2][4, 0, 7, 3, 1, 6, 2, 5][4, 0, 7, 5, 2, 6, 1, 3][4, 1, 3, 5, 7, 2, 0, 6][4, 1, 3, 6, 2, 7, 5, 0][4, 1, 5, 0, 6, 3, 7, 2][4, 1, 7, 0, 3, 6, 2, 5][4, 2, 0, 5, 7, 1, 3, 6][4, 2, 0, 6, 1, 7, 5, 3][4, 2, 7, 3, 6, 0, 5, 1][4, 6, 0, 2, 7, 5, 3, 1][4, 6, 0, 3, 1, 7, 5, 2][4, 6, 1, 3, 7, 0, 2, 5][4, 6, 1, 5, 2, 0, 3, 7][4, 6, 1, 5, 2, 0, 7, 3][4, 6, 3, 0, 2, 7, 5, 1][4, 7, 3, 0, 2, 5, 1, 6][4, 7, 3, 0, 6, 1, 5, 2][5, 0, 4, 1, 7, 2, 6, 3][5, 1, 6, 0, 2, 4, 7, 3][5, 1, 6, 0, 3, 7, 4, 2][5, 2, 0, 6, 4, 7, 1, 3][5, 2, 0, 7, 3, 1, 6, 4][5, 2, 0, 7, 4, 1, 3, 6][5, 2, 4, 6, 0, 3, 1, 7][5, 2, 4, 7, 0, 3, 1, 6][5, 2, 6, 1, 3, 7, 0, 4][5, 2, 6, 1, 7, 4, 0, 3][5, 2, 6, 3, 0, 7, 1, 4][5, 3, 0, 4, 7, 1, 6, 2][5, 3, 1, 7, 4, 6, 0, 2][5, 3, 6, 0, 2, 4, 1, 7][5, 3, 6, 0, 7, 1, 4, 2][5, 7, 1, 3, 0, 6, 4, 2][6, 0, 2, 7, 5, 3, 1, 4][6, 1, 3, 0, 7, 4, 2, 5][6, 1, 5, 2, 0, 3, 7, 4][6, 2, 0, 5, 7, 4, 1, 3][6, 2, 7, 1, 4, 0, 5, 3][6, 3, 1, 4, 7, 0, 2, 5][6, 3, 1, 7, 5, 0, 2, 4][6, 4, 2, 0, 5, 7, 1, 3][7, 1, 3, 0, 6, 4, 2, 5][7, 1, 4, 2, 0, 6, 3, 5][7, 2, 0, 5, 1, 4, 6, 3][7, 3, 0, 2, 5, 1, 6, 4]
```

#### Result

The **8-Queens Problem** was successfully solved using the **Backtracking** algorithm. The algorithm systematically placed queens on the chessboard while checking for conflicts and backtracked whenever an invalid position was encountered. It successfully found a valid arrangement in which no two queens attacked each other, demonstrating the efficiency of backtracking for solving constraint satisfaction problems.

#### 1c. Cryptarithmic Puzzle Solver

- **Aim:** Resolve verbal arithmetic expressions (e.g.,  $(SEND + MORE = MONEY)$ ) by identifying matching single-digit mapping variables.
- **Algorithm:**
  1. Extract all unique alphabetic characters from the expression strings.
  2. Ensure total distinct characters do not exceed 10.
  3. Generate letter-to-digit map sequences via exhaustive permutation tracking.
  4. Filter mappings where leading word indices are evaluationally assigned zero.

5. Compute numerical string evaluations; check if equation parameters balance exactly.

### Python Code

```
from itertools import permutations

def solve_cryptarithmic():
    words = ["SEND", "MORE", "MONEY"]
    chars = tuple(set("".join(words)))
    first_chars = {w[0] for w in words}

    for perm in permutations(range(10), len(chars)):
        mapping = dict(zip(chars, perm))
        if any(mapping[ch] == 0 for ch in first_chars):
            continue

        send = mapping['S']*1000 + mapping['E']*100 + mapping['N']*10 + mapping['D']
        more = mapping['M']*1000 + mapping['O']*100 + mapping['R']*10 + mapping['E']
        money = mapping['M']*10000 + mapping['O']*1000 + mapping['N']*100 +
        mapping['E']*10 + mapping['Y']

        if send + more == money:
            return mapping
    return None

print("Cryptarithmic Solution Mapping:", solve_cryptarithmic())
```

### Output

**Cryptarithmic Solution Mapping: {'Y': 2, 'M': 1, 'S': 9, 'N': 6, 'O': 0, 'E': 5, 'D': 7, 'R': 8}**

### Result

**The Cryptarithmic Puzzle was successfully solved using Python. The program assigned unique digits to letters while satisfying all arithmetic constraints and ensured that no two letters shared the same digit. The correct digit assignment was obtained, demonstrating the application of constraint satisfaction techniques in solving cryptarithmic problems.**

**Ex:No:2****Informed Search: A\* and Memory Bounded A\* (SMA\*)****2a. A\* Search**

- **Aim:** Find the absolute shortest structural node path balancing absolute graph path cost  $\backslash(g(n)\backslash)$  and estimated heuristic distance  $\backslash(h(n)\backslash)$ .
- **Algorithm:**
  1. Initialize a min-priority open queue containing structural parameters  $\backslash(f(n) = g(n) + h(n)\backslash)$ .
  2. Pop the node with the lowest  $\backslash(f(n)\backslash)$  value.
  3. If the pulled node is the goal state, track the absolute historical path.
  4. Iterate over neighboring structural elements.
  5. If a cheaper route to an adjacent state is found, update costs and push back into the priority queue.

**Python Code**

```
graph = {  
  
    'A': ['B', 'C'],  
  
    'B': ['D', 'E'],  
  
    'C': ['F', 'G'],  
  
    'D': ['H', 'J'],  
  
    'E': ['J', 'K'],  
  
    'F': ['L', 'M'],  
  
    'G': ['N', 'O'],  
  
    'H': [],  
  
    'I': [],  
  
    'J': [],
```

```
'K': [],  
  
'L': [],  
  
'M': [],  
  
'N': [],  
  
'O': []  
}  
  
def MA(start, goal, path, level, maxD):  
  
    print("\nCurrent level -->", level)  
  
    print("Goal node testing for", start)  
  
    path.append(start)  
  
    if start == goal:  
  
        print("Goal test successful")  
  
        return True  
  
    else:  
  
        print("Goal node test failed")  
  
    if level == maxD:  
  
        path.pop()  
  
        return False  
  
    print("\nExpanding the current node", start)
```

```
    for child in graph[start]:
        if MA(child, goal, path, level + 1, maxD):
            return True
    path.pop()
    return False

# Main Program

start = 'A'

goal = input("Enter the goal state: ")

maxD = int(input("Enter the maximum depth limit: "))

path = []

if MA(start, goal, path, 1, maxD):
    print("\nPath to goal node available")
    print("Path:", path)
else:
    print("\nNo path available for the goal node in the given depth limit")
```

### Output

**Enter the goal state: K**

**Enter the maximum depth limit: 4**

**Enter the goal state: 0 Enter the maximum depth limit: 3**

**Ex: No. 2(b): Informed Search – A and Memory-Bounded A (SMA\*)\*\*****Aim**

To implement **A\*** Search and **Simplified Memory-Bounded A\* (SMA\*)** algorithms using Python to find the optimal path from a start node to a goal node.

**A\* Search Algorithm**

A\* is an informed search algorithm that finds the shortest path between a start node and a goal node using the evaluation function:

$$\begin{aligned} &[ \\ &f(n) = g(n) + h(n) \\ &] \end{aligned}$$

Where:

- **g(n)** = Cost from the start node to the current node.
- **h(n)** = Heuristic estimate from the current node to the goal.
- **f(n)** = Estimated total cost.

**Memory-Bounded A\* (SMA\*)**

SMA\* (Simplified Memory-Bounded A\*) is an extension of A\* that works with limited memory. When the memory limit is reached, it removes the least promising node while retaining enough information to regenerate it if necessary.

**Algorithm****A\* Search**

1. Initialize the open list with the start node.
2. Initialize the closed list as empty.
3. Select the node with the lowest **f(n)**.
4. If the selected node is the goal, stop.
5. Expand the node and calculate **g**, **h**, and **f** values.
6. Add successors to the open list.
7. Repeat until the goal is found or no path exists.

**SMA\***

1. Initialize the open list with the start node.
2. Set a memory limit.
3. Expand the best node.
4. If memory exceeds the limit, remove the least promising node.
5. Continue until the goal is reached.
6. Display the optimal path.

**Python Program**

```
import heapq

# Graph with edge costs
graph = {
    'A': {'B': 1, 'C': 3},
    'B': {'D': 3, 'E': 1},
    'C': {'F': 5},
    'D': {'G': 2},
    'E': {'G': 4},
    'F': {'G': 1},
    'G': {}
}

# Heuristic values
heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 2,
    'E': 3,
    'F': 1,
    'G': 0
}

# A* Search Algorithm
```

```
def astar(start, goal):
    open_list = [(heuristic[start], 0, start, [start])]
    visited = set()

    while open_list:
        f, g, node, path = heapq.heappop(open_list)

        if node == goal:
            return path, g

        if node in visited:
            continue
        visited.add(node)

        for neighbor, cost in graph[node].items():
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]
                heapq.heappush(open_list, (new_f, new_g, neighbor, path + [neighbor]))

    return None, float('inf')

# Simplified Memory-Bounded A* (SMA*)
def sma_star(start, goal, memory_limit=3):
    open_list = [(heuristic[start], 0, start, [start])]
    visited = set()

    while open_list:
        if len(open_list) > memory_limit:
            open_list.pop() # Remove least promising node

        open_list.sort()

        f, g, node, path = open_list.pop(0)

        if node == goal:
            return path, g
```

```
    if node in visited:
        continue
    visited.add(node)

    for neighbor, cost in graph[node].items():
        if neighbor not in visited:
            new_g = g + cost
            new_f = new_g + heuristic[neighbor]
            open_list.append((new_f, new_g, neighbor, path + [neighbor]))

    return None, float('inf')

# Main Program
path, cost = astar('A', 'G')
print("A* Path:", path)
print("A* Cost:", cost)

path2, cost2 = sma_star('A', 'G')
print("\nSMA* Path:", path2)
print("SMA* Cost:", cost2)
```

### Sample Output

```
A* Path: ['A', 'B', 'E', 'G']
A* Cost: 6
```

```
SMA* Path: ['A', 'B', 'E', 'G']
SMA* Cost: 6
```

### Program Explanation

- A graph with edge costs and heuristic values is defined.
- The **A\*** algorithm selects the node with the smallest evaluation function  $f(n) = g(n) + h(n)$ .
- The **SMA\*** algorithm follows the same strategy but limits the number of nodes stored in memory.

- When the memory limit is exceeded, the least promising node is removed.
- Both algorithms find the shortest path from node **A** to node **G**.

### A\*

- Finds the optimal path.
- Efficient with good heuristics.
- Widely used in pathfinding.

### SMA\*

- Uses limited memory.
- Suitable for memory-constrained systems.
- Can handle large search spaces.

### Applications

- Robot navigation
- GPS route planning
- Game AI
- Network routing
- Autonomous vehicles

### Result

The A\* Search and **Simplified Memory-Bounded A\* (SMA\*)** algorithms were successfully implemented using Python. Both algorithms found the optimal path from the start node to the goal node. A\* efficiently used heuristic information to determine the shortest path, while SMA\* demonstrated memory-efficient search by limiting the number of stored nodes without significantly affecting the search outcome.

**EX.No. 3      Implement Mini-Max algorithm for game playing (Alpha-Beta pruning)**

**AIM** To Implement Mini-max algorithm for game playing.

**ALGORITHM**

1. The code first creates an array of five values.
2. These values represent the possible outcomes of a mini-max calculation.
3. Next, the code calls the minimax () function to find the optimal value for the current player.
4. This function takes three arguments: depth, nodeIndex, and maximizingPlayer.
5. The first two arguments specify how deep into the tree the minimax calculation should occur, and nodeIndex specifies which child node in that depth should be used as the starting point for this calculation.
6. The third argument is a Boolean flag that tells minimax () whether or not maximizing Player should be used during this calculation.
7. The code then sets up some variables to track which value in values represents best for each of MIN and MAX.
8. These variables are alpha and beta.
9. Alpha tracks which value in values is currently being optimized (MIN or MAX), while beta tracks which value is currently being minimized (MIN or MAX).
10. The code then loops through all of values' children nodes, calling minimax () on each one with different combinations of True and False as its arguments.
11. It keeps track of these results using best and val variables respectively.
12. If any of these calculations results in a better overall value than either MIN or MAX, then that new best value becomes the new minimum
13. The code will return the optimal value for the current player, which is 5

**Python Program**

```
MAX, MIN = 1000, -1000
# Returns optimal value for current player
def minimax(depth, nodeIndex, maximizingPlayer, values, alpha, beta):
    # Leaf node
    if depth == 3:
        return values[nodeIndex]
    if maximizingPlayer:
        best = MIN
        for i in range(2):
            val = minimax(depth + 1, nodeIndex * 2 + i,
```

```
        False, values, alpha, beta)
    best = max(best, val)
    alpha = max(alpha, best)
    # Alpha-Beta Pruning
    if beta <= alpha:
        break
    return best
else:
    best = MAX
    for i in range(2):
        val = minimax(depth + 1, nodeIndex * 2 + i,
                       True, values, alpha, beta)
        best = min(best, val)
        beta = min(beta, best)
        # Alpha-Beta Pruning
        if beta <= alpha:
            break
    return best
# Driver Code
if __name__ == "__main__":
    values = [3, 5, 6, 9, 1, 2, 0, -1]
    print("The optimal value is :", minimax(0, 0, True, values, MIN, MAX))
```

## OUTPUT

The optimal value  
is : 5

## Result

The **Mini-Max Algorithm with Alpha-Beta Pruning** was successfully implemented using Python.

**Ex:No:4****Solve constraint satisfaction problems**

**Aim:** To solve, the program uses backtracking to assign a day to each subject while ensuring that no two subjects connected by a constraint are scheduled on the same day.

**Algorithm**

1. The code starts by defining some variables.
2. The first variable, assignment, is a list of strings.
3. The second variable, VARIABLES, is a list of five strings.
4. Next, the code defines some constraints.
5. The first constraint is that the two lists have the same length ( $\text{len}(\text{assignment}) == \text{len}(\text{VARIABLES})$ ).
6. The next constraint is that each string in VARIABLES must be in one of the three domains (Monday, Tuesday, and Wednesday).
7. Finally, the last constraint is that each string in DOMAIN must be associated with at least one variable in VARIABLES.
8. The code then starts to search for an assignment that meets all the constraints.
9. First it checks if assignment is complete (if  $\text{len}(\text{assignment}) == \text{len}(\text{VARIABLES})$ ) and then it selects a variable from VARIABLES based on its domain and value.
10. If there are no more variables to select, then the code returns none.
11. Otherwise it backtracks through the assignment looking for a consistent combination of values for each selected variable.
12. If successful, it assigns those values to corresponding variables in result and returns this as solution.
13. The code first looks for a variable not yet assigned in the VARIABLES list.
14. If no variable is found, it returns none.
15. If an assignment is found, the code checks to see if the variables are consistent with each other.
16. If they are not, the code backtracks and tries again with a different variable.
17. If all variables are consistent with each other, then the code assigns the value of day to var3 and returns False

**Python Program**

```
# Variables
```

```
VARIABLES = ["csc", "maths", "phy", "che", "tam", "eng", "bio"]
```

```
# Domain (Available days)
```

```
DOMAIN = ["Monday", "Tuesday", "Wednesday"]
```

```
# Constraints (Subjects that cannot be on the same day)
```

```
CONSTRAINTS = [
```

```
    ("csc", "maths"),
```

```
    ("csc", "phy"),
```

```
    ("maths", "phy"),
```

```
    ("maths", "che"),
```

```
    ("maths", "tam"),
```

```
    ("phy", "tam"),
```

```
    ("phy", "eng"),
```

```
    ("che", "eng"),
```

```
    ("tam", "eng"),
```

```
    ("tam", "bio"),
```

```
    ("eng", "bio")
```

```
]
```

```
# Backtracking function
```

```
def backtrack(assignment):
```

```
    if len(assignment) == len(VARIABLES):
```

```
        return assignment
```

```
    var = select_unassigned_variable(assignment)
```

```
    for value in DOMAIN:
```

```
        if consistent(var, value, assignment):
```

```
            assignment[var] = value
```

```
            result = backtrack(assignment)
```

```
            if result is not None:
```

```
                return result
```

```
            del assignment[var]
```

```
    return None
```

```
# Select the next unassigned variable
```

```
def select_unassigned_variable(assignment):
```

```
    for var in VARIABLES:
```

```
    if var not in assignment:
        return var

# Check consistency with constraints
def consistent(var, value, assignment):
    for v1, v2 in CONSTRAINTS:
        if var == v1 or var == v2:
            other = v2 if var == v1 else v1
            if other in assignment and assignment[other] == value:
                return False
    return True

# Solve the CSP
solution = backtrack({})

# Display the result
if solution:
    print("Solution:")
    for subject, day in solution.items():
        print(subject, "->", day)
else:
    print("No solution exists.")
```

### Sample Output

```
Solution:
csc -> Monday
maths -> Tuesday
phy -> Wednesday
che -> Monday
tam -> Monday
eng -> Tuesday
bio -> Wednesday
```

### Result:

This program uses **backtracking** to assign a day to each subject while ensuring that no two subjects connected by a constraint are scheduled on the same day.

**Ex:No:5.                      Propositional Model Checking (TT-Entails)****Aim**

To implement **Propositional Model Checking Algorithms** using Python and verify whether a **Knowledge Base (KB)** entails a given query.

**Algorithm (Truth Table Entailment)**

1. Define the Knowledge Base (KB).
2. List all propositional symbols.
3. Generate all possible truth assignments.
4. Evaluate the KB for each assignment.
5. If the KB is true:
  - Check whether the query is also true.
6. If the query is true in every model satisfying the KB, return **True**.
7. Otherwise, return **False**.

**Python Program**

```
def evaluate_kb(models):  
    # Knowledge Base: P -> Q, P  
    return (not models['P'] or models['Q']) and models['P']  
  
def tt_entails(symbols, models, query):  
  
    if not symbols:  
        if evaluate_kb(models):  
            return models[query]  
        return True  
  
    p = symbols[0]  
    rest = symbols[1:]  
    return (tt_entails(rest, {**models, p: True}, query) and
```

```
tt_entails(rest, {**models, p: False}, query))
print("TT-Entails Evaluation (Does KB entail Q?):",
      tt_entails(['P', 'Q'], {}, 'Q'))
```

### Output

TT-Entails Evaluation (Does KB entail Q?): True

### Result

The Propositional Model Checking (TT-Entails) algorithm was successfully implemented in Python. The program generated all possible truth assignments for the propositional symbols, checked whether the Knowledge Base (KB) was true in each model, and verified if the given query was also true in all those models. Since the query was true in every model where the KB was true, it was concluded that the Knowledge Base entails the query ( $KB \models \text{Query}$ ). The implementation correctly demonstrated the working of the Truth Table Entailment (TT-Entails) algorithm for propositional logic.

**Ex:No:6      Implement Forward Chaining, Backward Chaining, and Resolution Strategies****Aim**

To implement **Forward Chaining, Backward Chaining, and Resolution** strategies in Python for propositional logic inference.

**1. Forward Chaining**

Forward Chaining is a **data-driven** inference method. It starts with the known facts and repeatedly applies inference rules to derive new facts until the goal is reached or no more inferences can be made.

**Algorithm**

1. Start with known facts.
2. Check all rules.
3. If all premises of a rule are true, infer the conclusion.
4. Add the conclusion to the known facts.
5. Repeat until the goal is found or no new facts can be inferred.

**Python Program**

```
def forward_chaining(facts, rules, goal):  
    inferred = set(facts)  
  
    while True:  
        new_fact = False  
  
        for premise, conclusion in rules:  
            if set(premise).issubset(inferred) and conclusion not in inferred:  
                inferred.add(conclusion)  
                new_fact = True  
  
        if goal in inferred:  
            return True  
  
        if not new_fact:  
            return False
```

```
facts = ['P']
rules = [
    (['P'], 'Q'),
    (['Q'], 'R')
]

goal = 'R'

print("Goal Found:", forward_chaining(facts, rules, goal))
```

### Output

Goal Found: True

## 2. Backward Chaining

Backward Chaining is a **goal-driven** inference method. It starts with the goal and works backward to determine whether the known facts can support it.

### Algorithm

1. Start with the goal.
2. If the goal is already a fact, return True.
3. Find rules whose conclusion matches the goal.
4. Recursively prove all premises.
5. If all premises are proved, return True.
6. Otherwise, return False.

### Python Program

```
def backward_chaining(facts, rules, goal):
    if goal in facts:
        return True

    for premise, conclusion in rules:
        if conclusion == goal:
            if all(backward_chaining(facts, rules, p) for p in premise):
                return True

    return False
```

```
facts = ['P']
rules = [
    (['P'], 'Q'),
    (['Q'], 'R')
]

goal = 'R'

print("Goal Found:", backward_chaining(facts, rules, goal))
```

### Output

Goal Found: True

### 3. Resolution Strategy

Resolution is a **proof by contradiction** technique.

Steps:

- Negate the query.
- Add it to the Knowledge Base.
- Apply the resolution rule.
- If the empty clause is obtained, the query is proved.

Example:

Knowledge Base

$P \rightarrow Q$   
 $P$

Convert to CNF

$\neg P \vee Q$   
 $P$

Negate Query

$\neg Q$

Resolve

$$\neg P \vee Q$$

$$\neg Q$$

-----

$$\neg P$$

$$P$$

$$\neg P$$

-----

$$\{\}$$

The empty clause  $\{\}$  is obtained.

Hence,

$$KB \models Q$$

### Python Program

```
def resolve(c1, c2):
    for literal in c1:
        if literal.startswith('-'):
            opposite = literal[1:]
        else:
            opposite = '-' + literal

        if opposite in c2:
            return (c1 - {literal}) | (c2 - {opposite})

    return None
```

```
clause1 = {'-P', 'Q'}
```

```
clause2 = {'P'}
```

```
result = resolve(clause1, clause2)
```

```
print("Resolved Clause:", result)
```

**Output**

Resolved Clause: {'Q'}

**Result:** The Forward Chaining, Backward Chaining, and Resolution strategies were successfully implemented using Python. The inference mechanisms correctly derived new facts, proved the specified goal, and resolved logical clauses, demonstrating the effectiveness of rule-based reasoning techniques in Artificial Intelligence.

**Ex:No:7.****Naïve Bayes Classifier**

- **Aim:** Predict categorical outcomes assuming features are conditionally independent given the class label.
- **Algorithm:**
  1. Calculate the prior probability  $P(C)$  for each class.
  2. Compute conditional probabilities  $P(X_i | C)$  for all features given each class.
  3. Multiply the class prior by the conditional probabilities of the observed features.
  4. Assign the sample to the class that yields the highest posterior probability.

**Python Code**

```
import numpy as np
```

```
class NaiveBayes:
```

```
    def fit(self, X, y):
```

```
        self.classes = np.unique(y)
```

```
        self.mean = np.array([X[y == c].mean(axis=0) for c in self.classes])
```

```
        self.var = np.array([X[y == c].var(axis=0) for c in self.classes])
```

```
        self.priors = np.array([X[y == c].shape[0] / float(X.shape[0]) for c in self.classes])
```

```
    def predict(self, X):
```

```
        preds = []
```

```
        for x in X:
```

```
            posteriors = []
```

```
            for idx, c in enumerate(self.classes):
```

```
                prior = np.log(self.priors[idx])
```

```
                conditional = np.sum(np.log(self._pdf(idx, x)))
```

```
                posteriors.append(prior + conditional)
```

```
            preds.append(self.classes[np.argmax(posteriors)])
```

```
        return preds
```

```
    def _pdf(self, class_idx, x):
```

```
        mean = self.mean[class_idx]
```

```
        var = self.var[class_idx]
```

```
        return np.exp(-((x - mean) ** 2) / (2 * var)) / np.sqrt(2 * np.pi * var)
```

```
X = np.array([[1, 2], [2, 3], [3, 4], [10, 11], [11, 12]])
y = np.array([0, 0, 0, 1, 1])
nb = NaiveBayes()
nb.fit(X, y)
print("Naive Bayes Classification:", nb.predict([[2, 2], [12, 13]]))
```

#### Output

Naive Bayes Classification: [np.int64(0), np.int64(1)]

**Result:** The **Naïve Bayes Classifier** was successfully implemented using Python. The classifier learned from the training data and correctly predicted the class labels for the given test data based on **Bayes' Theorem**.

**Ex:No:8.****Bayesian Networks & Inference****Aim**

To implement a Bayesian Network and perform probabilistic inference using Python.

A Bayesian Network (BN) is a probabilistic graphical model that represents variables and their conditional dependencies using a Directed Acyclic Graph (DAG).

**Each:**

- Node represents a random variable.
- Edge represents a dependency between variables.
- Conditional Probability Table (CPT) specifies the probability of a node given its parent nodes.

Bayesian Networks are used in:

- Medical diagnosis
- Weather forecasting
- Fault detection
- Decision support systems
- Risk analysis

Inference is the process of computing the probability of an unknown variable based on observed evidence.

**Algorithm**

1. Define the variables of the Bayesian Network.
2. Specify the conditional probability tables (CPTs).
3. Calculate the prior probabilities.
4. Apply Bayes' theorem to compute posterior probabilities.
5. Use the evidence to perform inference.
6. Display the probability of the queried event.

**Python Program (Simple Bayesian Inference)**

This program does not require any external libraries except NumPy, so it runs easily in Google Colab or any online Python interpreter.

*# Bayesian Network & Inference (Simple Example)*

*# Prior probabilities*

P\_Rain = 0.2

P\_NoRain = 0.8

*# Conditional probabilities*

P\_WetGrass\_given\_Rain = 0.9

P\_WetGrass\_given\_NoRain = 0.1

*# Total probability of Wet Grass*

```
P_WetGrass = (  
    P_WetGrass_given_Rain * P_Rain +  
    P_WetGrass_given_NoRain * P_NoRain  
)
```

*# Bayes' Theorem*

```
P_Rain_given_WetGrass = (  
    P_WetGrass_given_Rain * P_Rain  
) / P_WetGrass
```

```
print("Probability of Rain =", P_Rain)
```

```
print("Probability of Wet Grass =", round(P_WetGrass, 2))
```

```
print("Probability of Rain given Wet Grass =", round(P_Rain_given_WetGrass, 2))
```

Sample Output

Probability of Rain = 0.2

Probability of Wet Grass = 0.26

Probability of Rain given Wet Grass = 0.69

**Result:** The **Bayesian Network and Inference** were successfully implemented using Python. The Bayesian Network represented the probabilistic relationships between variables, and inference was performed using **Bayes' Theorem** to compute the posterior probability based on the given evidence.

**Ex:No:9.****Goal-Based Autonomous Agent****Aim**

To implement a **Goal-Based Autonomous Agent** using Python that selects actions to achieve a specified goal.

A **Goal-Based Agent** is an intelligent agent that chooses actions based on a predefined goal. Unlike a simple reflex agent, it considers the future consequences of its actions and decides the best path to reach the goal.

**Characteristics**

- Has a current state.
- Has a goal state.
- Selects actions that move it toward the goal.
- Stops when the goal is reached.

**Applications**

- Robot navigation
- GPS route planning
- Game AI
- Automated decision-making

**Algorithm**

1. Initialize the current state and goal state.
2. Compare the current state with the goal.
3. If the current state is not the goal, move one step toward the goal.
4. Update the current state.
5. Repeat Steps 2–4 until the goal is reached.
6. Display the path taken and the final state.

**Python Program**

```
# Goal-Based Autonomous Agent
```

```
def goal_based_agent(start, goal):
```

```
    current = start
```

```
print("Start State :", current)

print("Goal State :", goal)

print("\nPath:")

while current != goal:

    print(current, end=" -> ")

    if current < goal:

        current += 1

    else:

        current -= 1

print(goal)

print("\nGoal Reached Successfully!")

# Main Program

start = int(input("Enter Start State: "))

goal = int(input("Enter Goal State: "))

goal_based_agent(start, goal)
```

**Sample Input**

Enter Start State: 2

Enter Goal State: 7

**Output**

Start State : 2

Goal State : 7

Path:

2 -> 3 -> 4 -> 5 -> 6 -> 7

Goal Reached Successfully!

### Another Example

#### Input

Enter Start State: 8

Enter Goal State: 3

#### Output

Start State : 8

Goal State : 3

Path:

8 -> 7 -> 6 -> 5 -> 4 -> 3

Goal Reached Successfully!

### Program Explanation

- The user enters the **start state** and the **goal state**.
- The agent checks whether the current state matches the goal.
- If the current state is smaller than the goal, it increments the state by 1.
- If the current state is greater than the goal, it decrements the state by 1.
- The process continues until the goal is reached.
- Finally, the complete path and a success message are displayed.

### Result

The Goal-Based Autonomous Agent was successfully implemented in Python. The agent achieved the specified goal by selecting appropriate actions and displayed the path from the start state to the goal state.

**Ex: No. 10****Reflex Agent & Model-Based Agent****Aim**

To implement **Reflex Agent** and **Model-Based Agent** using Python and demonstrate their decision-making process based on the environment.

**Reflex Agent**

A **Reflex Agent** selects actions based only on the **current percept (input)**. It follows predefined condition-action rules and does not remember past states.

**Characteristics:**

- Uses current input only.
- Fast and simple.
- Suitable for fully observable environments.

**Model-Based Agent**

A **Model-Based Agent** maintains an **internal model (memory)** of the environment. It updates its knowledge based on previous states and current observations before selecting an action.

**Characteristics:**

- Maintains internal state.
- Suitable for partially observable environments.
- Makes more informed decisions.

**Algorithm****Reflex Agent**

1. Read the current environment state.
2. Check the condition using predefined rules.
3. Select the corresponding action.
4. Display the action.

**Model-Based Agent**

1. Read the current environment state.
2. Update the internal model (memory).
3. Use the model to determine the appropriate action.
4. Display the updated state and action.

**Python Program**

```
# Reflex Agent and Model-Based Agent
```

```
# Reflex Agent
```

```
def reflex_agent(room_status):
```

```
    if room_status == "Dirty":
```

```
        return "Clean the room"
```

```
    else:
```

```
        return "No action needed"
```

```
# Model-Based Agent
```

```
class ModelBasedAgent:
```

```
    def __init__(self):
```

```
        self.memory = {}
```

```
    def update(self, room, status):
```

```
        self.memory[room] = status
```

```
    def act(self, room):
```

```
        if self.memory[room] == "Dirty":
```

```
            return "Clean the room"
```

```
        else:

            return "Room is already clean"

# Main Program

print("----- Reflex Agent -----")

status = input("Enter room status (Clean/Dirty): ")

print("Action:", reflex_agent(status))

print("\n----- Model-Based Agent -----")

agent = ModelBasedAgent()

room = input("Enter room name (A/B): ")

status = input("Enter room status (Clean/Dirty): ")

agent.update(room, status)

print("Memory:", agent.memory)

print("Action:", agent.act(room))
```

### Sample Input

Enter room status (Clean/Dirty): Dirty

Enter room name (A/B): A

Enter room status (Clean/Dirty): Clean

### Sample Output

----- Reflex Agent -----

Enter room status (Clean/Dirty): Dirty

Action: Clean the room

----- Model-Based Agent -----

Enter room name (A/B): A

Enter room status (Clean/Dirty): Clean

Memory: {'A': 'Clean'}

Action: Room is already clean

### Program Explanation

- The **Reflex Agent** checks the current room status.
  - If the room is **Dirty**, it chooses "**Clean the room**".
  - Otherwise, it performs **no action**.
- The **Model-Based Agent** stores the room status in its internal memory.
  - It updates the memory whenever a room status is provided.
  - It uses the stored information to decide whether cleaning is required.

### Reflex Agent

- Simple and fast.
- Easy to implement.
- Suitable for simple environments.

### Model-Based Agent

- Maintains memory of previous states.
- Works well in partially observable environments.
- Makes better decisions than a reflex agent.

### Result

The **Reflex Agent** and **Model-Based Agent** were successfully implemented using Python. The Reflex Agent selected actions based only on the current percept, while the Model-Based Agent maintained an internal memory of the environment and used it to make informed decisions. Both agents behaved as expected according to their respective decision-making strategies.

**Ex:No:11. Environment-Aware Intelligent Agent****Aim**

To implement an **Environment-Aware Intelligent Agent** using Python that senses the environment and performs appropriate actions based on the current environmental conditions.

An **Environment-Aware Intelligent Agent** is an intelligent system that continuously observes its surroundings (environment), interprets the current state, and selects suitable actions. Unlike a simple reflex agent, it can adapt its behavior according to different environmental conditions.

**Characteristics**

- Senses the environment.
- Analyzes the current situation.
- Takes appropriate actions.
- Adapts to changing conditions.

**Applications**

- Smart home automation
- Autonomous vehicles
- Industrial automation
- Healthcare monitoring
- Smart agriculture

**Algorithm**

1. Read the current environmental condition.
2. Compare the condition with predefined rules.
3. Select the appropriate action.
4. Display the action taken by the agent.
5. Repeat for different environmental conditions if required.

**Python Program**

```
# Environment-Aware Intelligent Agent
```

```
def environment_agent(condition):
```

```
if condition.lower() == "rain":  
    return "Take an Umbrella"  
elif condition.lower() == "sunny":  
    return "Wear Sunglasses"  
elif condition.lower() == "cold":  
    return "Wear a Jacket"  
elif condition.lower() == "hot":  
    return "Drink Water and Stay Cool"  
else:  
    return "No Specific Action"
```

```
# Main Program
```

```
condition = input("Enter the Environment (Rain/Sunny/Cold/Hot): ")
```

```
action = environment_agent(condition)
```

```
print("Environment :", condition)
```

```
print("Agent Action:", action)
```

### Sample Input

Enter the Environment (Rain/Sunny/Cold/Hot): Rain

### Sample Output

Environment : Rain

Agent Action: Take an Umbrella

### Another Example

#### Input

Enter the Environment (Rain/Sunny/Cold/Hot): Cold

#### Output

Environment : Cold

Agent Action: Wear a Jacket

### Program Explanation

- The program accepts the current environmental condition as input.
- The agent compares the input with predefined rules using if-elif statements.
- Based on the condition:
  - **Rain** → Take an Umbrella
  - **Sunny** → Wear Sunglasses
  - **Cold** → Wear a Jacket
  - **Hot** → Drink Water and Stay Cool
- If the condition does not match any predefined rule, the agent performs **No Specific Action**.
- Finally, the selected action is displayed.

### Advantages

- Simple and easy to implement.
- Responds to environmental changes.
- Improves decision-making.
- Useful in real-time intelligent systems.

### Applications

- Smart homes
- Weather-based recommendation systems
- Robotics
- IoT devices
- Intelligent assistants

### Result

The **Environment-Aware Intelligent Agent** was successfully implemented using Python. The agent sensed the environmental condition and selected the appropriate action based on predefined rules, demonstrating adaptive decision-making in response to different environmental situations.

**Ex:No:12. Learning Agent with Memory Update****Aim**

To implement a **Learning Agent with Memory Update** using Python that learns from user interactions by storing previous experiences and updating its memory to improve future decisions.

A **Learning Agent** is an intelligent agent that improves its performance by learning from past experiences. It stores information about previous interactions in its **memory** and uses this knowledge to make better decisions when similar situations occur.

**Components of a Learning Agent**

- **Performance Element:** Chooses the action.
- **Learning Element:** Updates knowledge from experience.
- **Memory:** Stores previous observations and actions.
- **Environment:** Provides inputs to the agent.

**Applications**

- Chatbots
- Recommendation systems
- Virtual assistants
- Robotics
- Personalized learning systems

**Algorithm**

1. Initialize an empty memory.
2. Read the situation (input) from the user.
3. Read the action associated with the situation.
4. Store the situation and action in memory.
5. Display the updated memory.
6. If the same situation is entered again, retrieve and display the stored action.
7. Repeat until the user chooses to exit.

**Python Program**

```
# Learning Agent with Memory Update
class LearningAgent:
    def __init__(self):
        self.memory = {}
    def learn(self, situation, action):
        self.memory[situation] = action
        print("Memory Updated!")
    def recall(self, situation):
        if situation in self.memory:
            print("Known Action:", self.memory[situation])
        else:
            print("Situation not found in memory.")

# Main Program
agent = LearningAgent()
while True:
    print("\n1. Learn")
    print("2. Recall")
    print("3. Exit")
    choice = input("Enter your choice: ")
    if choice == "1":
        situation = input("Enter Situation: ")
        action = input("Enter Action: ")
        agent.learn(situation, action)
    elif choice == "2":
        situation = input("Enter Situation: ")
        agent.recall(situation)

    elif choice == "3":
        print("Exiting...")
```

```
        break
    else:
        print("Invalid Choice")
```

### **Sample Input and Output**

1. Learn

2. Recall

3. Exit

Enter your choice: 1

Enter Situation: Rain

Enter Action: Take Umbrella

Memory Updated!

1. Learn

2. Recall

3. Exit

Enter your choice: 2

Enter Situation: Rain

Known Action: Take Umbrella

1. Learn

2. Recall

3. Exit

Enter your choice: 3

Exiting...

### Program Explanation

- A class LearningAgent is created with a dictionary called memory.
- The learn() method stores a situation and its corresponding action in memory.
- The recall() method checks whether the situation exists in memory:
  - If found, it displays the stored action.
  - Otherwise, it informs the user that the situation is unknown.
- The menu-driven program allows the user to repeatedly learn new situations, recall learned actions, or exit.

### Advantages

- Learns from previous experiences.
- Updates memory dynamically.
- Improves decision-making over time.
- Easy to implement and extend.

### Applications

- Intelligent tutoring systems
- Chatbots
- Smart assistants
- Autonomous robots
- Decision support systems

### Result

The **Learning Agent with Memory Update** was successfully implemented using Python. The agent learned new situations by storing them in memory, updated its knowledge dynamically, and successfully recalled the appropriate actions for previously learned situations.

**Ex:No:13. Cooperative Multi-Agent Communication System****Aim**

To implement a **Cooperative Multi-Agent Communication System** using Python, where multiple agents communicate and cooperate to complete a common task.

A **Multi-Agent System (MAS)** consists of two or more intelligent agents that interact with one another to achieve individual or shared goals. In a **Cooperative Multi-Agent System**, agents exchange information and coordinate their actions to solve a problem efficiently.

**Characteristics**

- Multiple autonomous agents.
- Communication between agents.
- Cooperation to achieve a common goal.
- Coordination and information sharing.

**Applications**

- Robot teams
- Traffic management
- Smart grids
- Distributed computing
- Supply chain management

**Algorithm**

1. Create two or more agents.
2. Assign a message to be communicated.
3. Agent 1 sends the message to Agent 2.
4. Agent 2 receives the message and sends an acknowledgment.
5. Display the communication between the agents.
6. End the communication.

**Python Program**

```
# Cooperative Multi-Agent Communication System
```

```
class Agent:
    def __init__(self, name):
        self.name = name

    def send_message(self, receiver, message):
        print(f'{self.name} sends to {receiver.name}: {message}')
        receiver.receive_message(self.name, message)

    def receive_message(self, sender, message):
        print(f'{self.name} received from {sender}: {message}')
        print(f'{self.name}: Task acknowledged.\n")

# Main Program
agent1 = Agent("Agent A")
agent2 = Agent("Agent B")

message = input("Enter the message to communicate: ")

agent1.send_message(agent2, message)
```

### Sample Input

Enter the message to communicate: Complete the assigned task

### Sample Output

Agent A sends to Agent B: Complete the assigned task  
Agent B received from Agent A: Complete the assigned task  
Agent B: Task acknowledged.

### Program Explanation

- A class Agent is created with a **name** attribute.
- The send\_message() method sends a message from one agent to another.
- The receiving agent processes the message using the receive\_message() method.
- After receiving the message, the second agent acknowledges the task.
- This demonstrates simple communication and cooperation between intelligent agents.

### Advantages

- Enables cooperation among multiple agents.
- Supports distributed problem solving.
- Improves task efficiency.
- Easy to extend for multiple agents.

### Applications

- Autonomous robots
- Smart transportation systems
- Distributed AI systems
- Network communication
- Collaborative decision-making

### Result

The **Cooperative Multi-Agent Communication System** was successfully implemented using Python. The agents communicated by exchanging messages and cooperated to accomplish a common task, demonstrating the basic principles of multi-agent communication and coordination.

**Ex:No:14. Trust & Reputation System (Online Rating Integration)****Aim**

To implement a **Trust & Reputation System** using Python that collects user ratings, calculates the average rating, and determines the reputation of an entity based on the received ratings.

A **Trust & Reputation System** is used in online platforms to evaluate the reliability and quality of users, products, or services. It collects ratings from multiple users and computes a reputation score that helps others make informed decisions.

**Characteristics**

- Collects ratings from multiple users.
- Calculates an overall reputation score.
- Updates reputation dynamically.
- Assists in decision-making.

**Applications**

- E-commerce websites
- Online marketplaces
- Hotel and restaurant review systems
- Ride-sharing applications
- Freelancing platforms

**Algorithm**

1. Read the number of users providing ratings.
2. Accept the rating from each user.
3. Calculate the average rating.
4. Determine the reputation based on the average rating:
  - $\text{Average} \geq 4.5 \rightarrow \text{Excellent}$
  - $\text{Average} \geq 3.5 \rightarrow \text{Good}$
  - $\text{Average} \geq 2.5 \rightarrow \text{Average}$
  - Otherwise  $\rightarrow \text{Poor}$
5. Display the average rating and reputation.

**Python Program**

```
# Trust & Reputation System (Online Rating Integration)
n = int(input("Enter the number of users: "))
ratings = []
for i in range(n):
    rating = float(input(f"Enter rating from User {i+1} (1-5): "))
    ratings.append(rating)
average = sum(ratings) / n
if average >= 4.5:
    reputation = "Excellent"
elif average >= 3.5:
    reputation = "Good"
elif average >= 2.5:
    reputation = "Average"
else:
    reputation = "Poor"
print("\nRatings:", ratings)
print("Average Rating:", round(average, 2))
print("Reputation:", reputation)
```

**Sample Input**

```
Enter the number of users: 5
Enter rating from User 1 (1-5): 5
Enter rating from User 2 (1-5): 4
Enter rating from User 3 (1-5): 5
Enter rating from User 4 (1-5): 4
Enter rating from User 5 (1-5): 5
```

### Sample Output

Ratings: [5.0, 4.0, 5.0, 4.0, 5.0]

Average Rating: 4.6

Reputation: Excellent

### Program Explanation

- The program first asks for the number of users.
- It collects ratings (between 1 and 5) from each user.
- The average rating is calculated using:

$\text{Average} = \text{Sum of Ratings} / \text{Number of Ratings}$

- Based on the average, the system assigns a reputation:
  - **Excellent** (4.5–5.0)
  - **Good** (3.5–4.49)
  - **Average** (2.5–3.49)
  - **Poor** (Below 2.5)
- Finally, it displays all ratings, the average rating, and the reputation.
- Simple and easy to implement.
- Helps users identify reliable products or services.
- Dynamically updates reputation as new ratings are added.
- Supports informed decision-making.

### Applications

- Amazon product ratings, Google Play Store reviews, Hotel booking websites
- Food delivery applications
- Online learning platforms

### Result

The **Trust & Reputation System** was successfully implemented using Python. The program collected user ratings, computed the average rating, and assigned an appropriate reputation level based on the calculated score, demonstrating the basic concept of online trust and reputation management.

**Ex:No:15. Distributed Problem-Solving Agents****Aim**

To implement **Distributed Problem-Solving Agents** using Python, where multiple agents cooperate by solving different parts of a problem and combining their results to produce the final solution.

A **Distributed Problem-Solving (DPS) System** consists of multiple intelligent agents working together to solve a complex problem. Instead of a single agent handling the entire task, each agent is responsible for a specific subtask. The partial results from all agents are then combined to obtain the final solution.

**Characteristics**

- Multiple autonomous agents.
- Task distribution among agents.
- Cooperation and coordination.
- Efficient use of resources.
- Parallel execution of subtasks.

**Applications**

- Distributed computing
- Cloud computing
- Robotics
- Traffic management systems
- Sensor networks
- Large-scale optimization problems

**Algorithm**

1. Divide the main problem into smaller subtasks.
2. Assign each subtask to an agent.
3. Each agent independently solves its assigned task.
4. Collect the results from all agents.
5. Combine the partial results.
6. Display the final solution.

**Python Program**

```
# Distributed Problem-Solving Agents
```

```
class Agent:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def solve(self, numbers):
```

```
        result = sum(numbers)
```

```
        print(f'{self.name} processed {numbers} -> Sum = {result}')
```

```
        return result
```

```
# Main Program
```

```
numbers = [10, 20, 30, 40, 50, 60]
```

```
# Divide the task into two parts
```

```
part1 = numbers[:3]
```

```
part2 = numbers[3:]
```

```
agent1 = Agent("Agent A")
```

```
agent2 = Agent("Agent B")
```

```
# Agents solve their subtasks
```

```
result1 = agent1.solve(part1)
```

```
result2 = agent2.solve(part2)
```

```
# Combine results
```

```
final_result = result1 + result2
```

```
print("\nFinal Sum =", final_result)
```

**Sample Output**

```
Agent A processed [10, 20, 30] -> Sum = 60
```

```
Agent B processed [40, 50, 60] -> Sum = 150
```

```
Final Sum = 210
```

### Program Explanation

- A class Agent is created with a solve() method.
- The input list is divided into two smaller parts.
- **Agent A** calculates the sum of the first part.
- **Agent B** calculates the sum of the second part.
- The partial sums are combined to obtain the final result.
- This demonstrates how multiple agents cooperate to solve a problem efficiently.
- Reduces computation time by dividing tasks.
- Supports parallel processing.
- Improves scalability.
- Increases reliability through cooperation.

### Applications

- Cloud computing
- Multi-robot systems
- Big data processing
- Distributed databases
- Scientific computing
- Smart grid management

### Result

The **Distributed Problem-Solving Agents** were successfully implemented using Python. The problem was divided into subtasks, solved independently by multiple agents, and the partial results were combined to produce the final solution, demonstrating the concept of distributed problem solving in intelligent systems.